IMPORT RELEVANT PYTHON LYBRARIES

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
```

In [2]:
```
warnings.filterwarnings('ignore')
```

IMPORT THE DATASET AND STUDY IT

In [3]:
```
df = pd.read_csv('Marketing_ROI_Visibility_Project_7000_Rows2.csv')
```

In [4]:
```
df.head()
```

Out[4]:

	CustomerID	Date	CampaignName	MarketingChannel	Product	Region	Impressions	Clic
0	CUST-100000	1/20/2025	Retention Drive	YouTube	Analytics Hub	Europe	33232	19
1	CUST-100001	1/1/2025	Retention Drive	Facebook Ads	Customer Insights Platform	North America	47933	34
2	CUST-100002	2/17/2025	Black Friday Promo	Instagram	CRM Pro	Asia Pacific	16504	21
3	CUST-100003	6/25/2025	High Intent Leads	Instagram	Marketing Automation Suite	Africa	84979	114
4	CUST-100004	8/11/2025	High Intent Leads	Instagram	Customer Insights Platform	Middle East	139414	217

DATA CLEANING

Remove Duplicates

In [5]:
```
df = df.drop_duplicates()
```

Fill Missing Values

In [6]:
```
df["Revenue"] = df["Revenue"].fillna(df["Revenue"].median())

df["MarketingCost"] = df["MarketingCost"].fillna(df["MarketingCost"].median())

df["CLV"] = df["CLV"].fillna(df["CLV"].median())
```

Fix Text Inconsistencies

In [7]:
```
df["MarketingChannel"] = (
    df["MarketingChannel"]
    .str.title()
    .replace({
        "Google Ads": "Google Ads",
        "Facebook Ads": "Facebook Ads"
    })
)
```

6 MOST IMPORTANT KPIs

TOTAL REVENUE

In [8]:
```
total_revenue = df["Revenue"].sum()

print("Total Revenue:", total_revenue)
```

```
Total Revenue: 1081622171
```

TOTAL MARKETING COST

In [9]:
```
total_cost = df["MarketingCost"].sum()

print("Total Marketing Cost:", total_cost)
```

```
Total Marketing Cost: 643239218.86
```

TOTAL PROFIT

In [10]:
```
profit = total_revenue - total_cost

print("Total Profit:", profit)
```

```
Total Profit: 438382952.14
```

ROI %

Formula

ROI%= Marketing Cost Revenue−Marketing Cost

×100

In [11]:
```
roi = ((total_revenue - total_cost) / total_cost) * 100

print("ROI %:", round(roi, 2))
```

```
ROI %: 68.15
```

CAC

Formula

CAC= Conversions Marketing Cost

In [12]:
```
cac = (
    df["MarketingCost"].sum()
    / df["Conversions"].sum()
)

print("CAC:", round(cac, 2))
```

```
CAC: 815.59
```

Conversion Rate

Formula

Conversion Rate= Leads Conversions

×100

In [13]:
```
conversion_rate = (
    df["Conversions"].sum()
    / df["Leads"].sum()
) * 100

print("Conversion Rate:", round(conversion_rate, 2))
```

```
Conversion Rate: 17.9
```

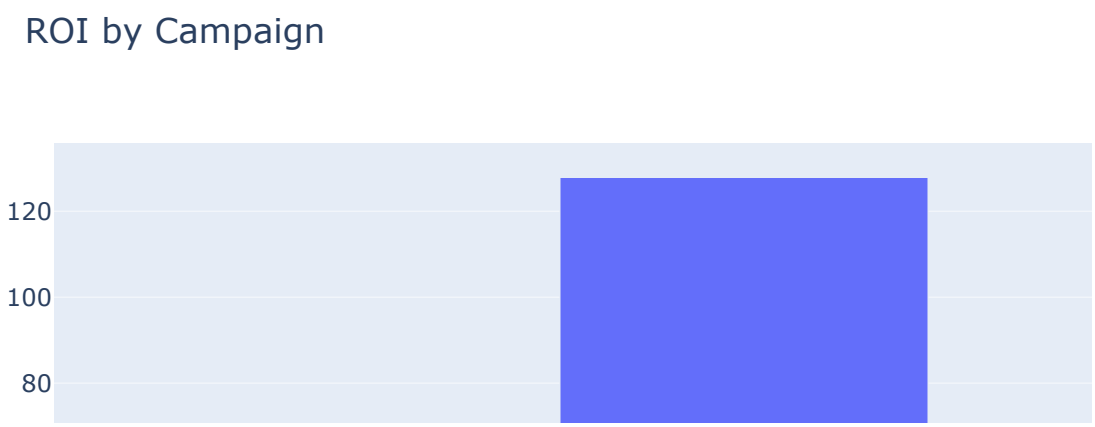
6 MOST IMPORTANT CHARTS

In [14]:
```
import pandas as pd
import numpy as np
import plotly.express as px
import matplotlib.pyplot as plt
```

CHANNEL BY ROI

In [15]:
```
campaign_roi = df.groupby("MarketingChannel").agg({
    "Revenue":"sum",
    "MarketingCost":"sum"
})

campaign_roi["ROI"] = (
    (
        campaign_roi["Revenue"]
        - campaign_roi["MarketingCost"]
```

```
    )
    / campaign_roi["MarketingCost"]
) * 100

campaign_roi = campaign_roi.reset_index()

fig = px.bar(
    campaign_roi,
    x="MarketingChannel",
    y="ROI",
    title="ROI by Campaign"
)

fig.show()
```



Purpose

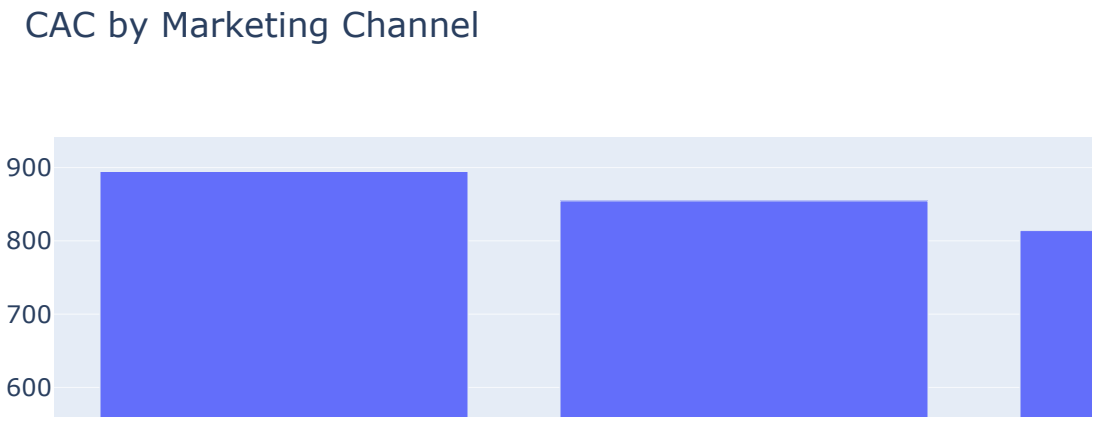
Identifies profitable and inefficient campaigns.

Executive Insight

Some campaigns generate high revenue but weak profitability.

CHANNEL BY CAC

In [16]:
```
channel_cac = df.groupby("MarketingChannel").agg({
    "MarketingCost":"sum",
    "Conversions":"sum"
})

channel_cac["CAC"] = (
    channel_cac["MarketingCost"]
    / channel_cac["Conversions"]
)

channel_cac = channel_cac.reset_index()

fig = px.bar(
    channel_cac,
    x="MarketingChannel",
    y="CAC",
    title="CAC by Marketing Channel"
)

fig.show()
```



Executive Insight

High CAC channels reduce acquisition efficiency.

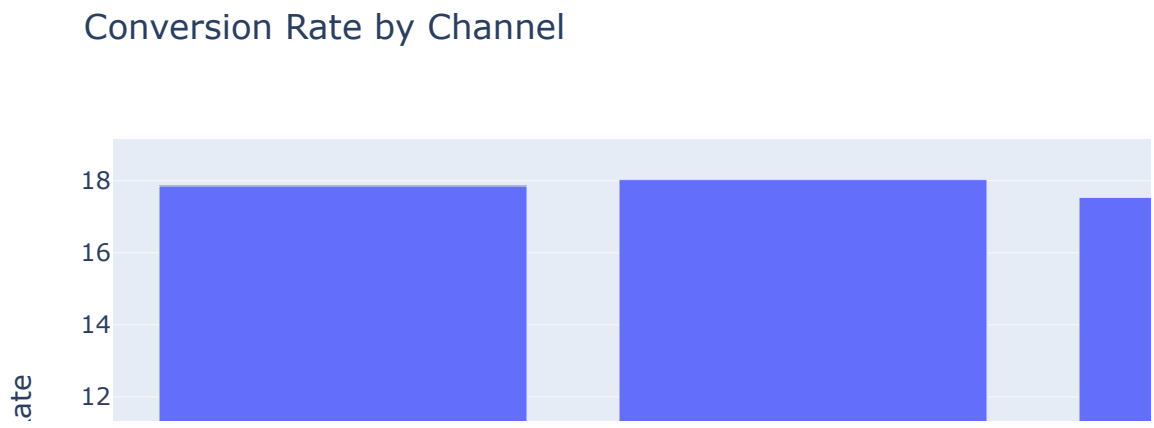
Conversion Rate by Channel Purpose

Shows which channels convert effectively.

CONVERSION BY CHANNEL

In [17]:
```
channel_conversion = df.groupby("MarketingChannel").agg({
    "Conversions":"sum",
    "Leads":"sum"
})

channel_conversion["ConversionRate"] = (
    channel_conversion["Conversions"]
    / channel_conversion["Leads"]
) * 100

channel_conversion = channel_conversion.reset_index()
```

```
fig = px.bar(
    channel_conversion,
    x="MarketingChannel",
    y="ConversionRate",
    title="Conversion Rate by Channel"
)

fig.show()
```



Executive Insight

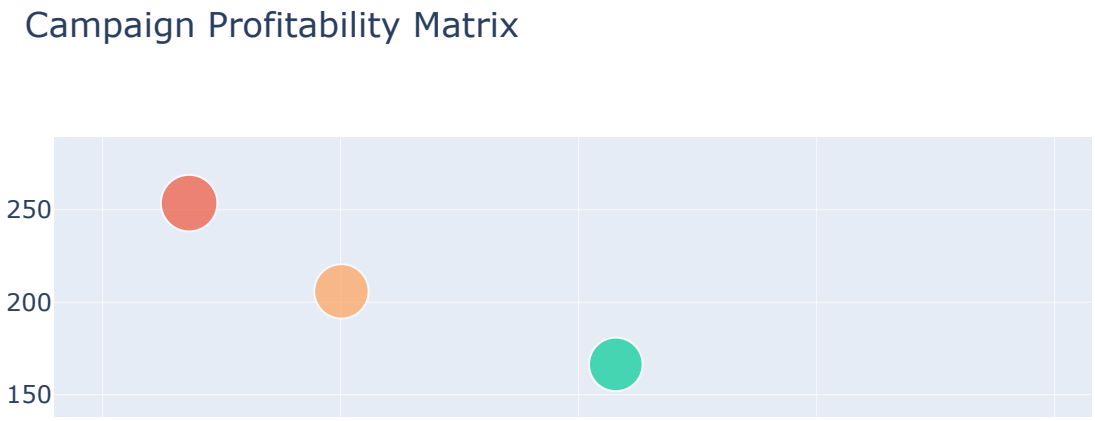
High CAC channels reduce acquisition efficiency.

Conversion Rate by Channel Purpose

Shows which channels convert effectively.

In [18]:
```
campaign_profitability = df.groupby("CampaignName").agg({
    "MarketingCost":"sum",
    "Revenue":"sum"
})

campaign_profitability["ROI"] = (
    (
```

```
        campaign_profitability["Revenue"]
        - campaign_profitability["MarketingCost"]
    )
    / campaign_profitability["MarketingCost"]
) * 100

campaign_profitability = campaign_profitability.reset_index()

fig = px.scatter(
    campaign_profitability,
    x="MarketingCost",
    y="ROI",
    size="Revenue",
    color="CampaignName",
    title="Campaign Profitability Matrix"
)

fig.show()
```



Executive Insight

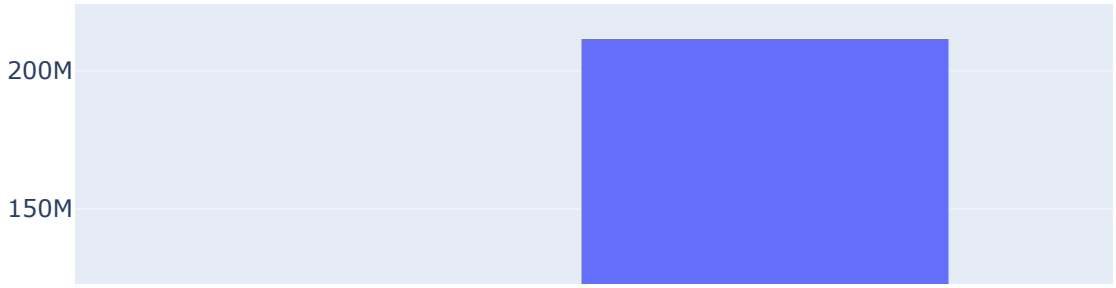
This chart identifies:

wasteful campaigns, scalable campaigns, and budget optimization opportunities. Profit by Marketing Channel Purpose

Shows which channels generate actual profitability.

Profit by Marketing Channel

In [19]:
```
channel_profit = df.groupby("MarketingChannel").agg({
    "Revenue":"sum",
    "MarketingCost":"sum"
})

channel_profit["Profit"] = (
    channel_profit["Revenue"]
    - channel_profit["MarketingCost"]
)

channel_profit = channel_profit.reset_index()

fig = px.bar(
    channel_profit,
    x="MarketingChannel",
    y="Profit",
    title="Profit by Marketing Channel"
)

fig.show()
```



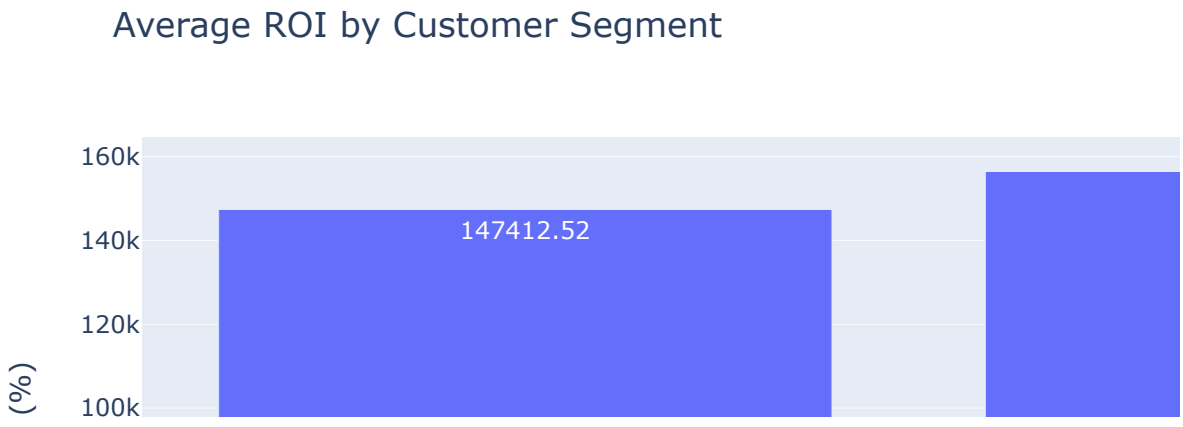
In [39]:
```python
def customer_segment(row):

    # High Value Customer
    if (
        row["CLV"] >= 15000
        and row["RetentionRate"] >= 0.75
        and row["ROI_Percentage"] > 0
    ):
        return "High Value Customer"

    # Risk Customer
    elif (
        row["ChurnRate"] >= 0.40
        or row["RetentionRate"] <= 0.40
    ):
        return "Risk Customer"

    # High Cost Customer
    elif (
        row["CAC"] >= 1000
        and row["ROI_Percentage"] <= 0

    ):
        return "High Cost Customer"

    # Mid Value Customer
```

```
    else:
        return "Mid Value Customer"

# Apply segmentation
df["CustomerSegment"] = df.apply(customer_segment, axis=1)
```

In [38]:

```
segment_roi = (
    df.groupby("CustomerSegment")["ROI_Percentage"]
    .mean()
    .reset_index()
)

# ========================================
# PRINT RESULTS
# ========================================

print(segment_roi)

# ========================================
# CREATE ROI CHART
# ========================================

fig = px.bar(
    segment_roi,
    x="CustomerSegment",
    y="ROI_Percentage",
    title="Average ROI by Customer Segment",
    text_auto=".2f"
)

fig.update_layout(
    xaxis_title="Customer Segment",
    yaxis_title="Average ROI (%)"
)

fig.show()
```

```
       CustomerSegment  ROI_Percentage
0  High Value Customer   147412.523717
1   Mid Value Customer   156472.516867
2         Risk Customer   150452.709322
```



EXPECTED BUSINESS INSIGHT

INTERPRETATION

If:

High Value Customers

have the highest ROI, this means:

retention, CLV, and acquisition quality

are driving profitability.

If:

High Cost Customers

have negative ROI, this indicates:

acquisition inefficiency, budget waste, or poor targeting.

Executive Insight

Some channels generate traffic but weak profitability.

EXECUTIVE STORYTELLING What Happened?

Marketing costs increased significantly while profitability weakened.

Why Did It Happen?

Root causes:

rising CAC,

Consultant-Level Final Conclusion

The project transforms marketing analytics from activity monitoring into profitability intelligence, enabling the organization to optimize acquisition strategy, improve marketing efficiency, reduce profitability leakage, and drive more sustainable long-term growth.

Executive-Level Closing Statement By aligning marketing investment decisions with customer profitability, acquisition efficiency, and long-term value creation, the organization can significantly improve financial performance while building a more scalable and sustainable growth model. This is the type of conclusion used in: • executive consulting reports, • strategy presentations, • and senior analyst case studies.

In [ ]:

MACHINE LEARNING

Marketing Campaign ROI Optimization & Profitability Prediction

Target Variable

ROI_Percentage

Features

Use:

MarketingChannel CampaignName Leads Conversions CAC CLV RetentionRate Region CustomerSegment CTR

Business Value

The company can:

predict profitable campaigns, reduce waste, optimize budget allocation.

In [20]:
```
pip install pandas numpy scikit-learn xgboost matplotlib seaborn joblib openpyxl
```

```
Requirement already satisfied: pandas in c:\users\hp\downloads\new folder\lib\site-pa
ckages (1.4.4)
Requirement already satisfied: numpy in c:\users\hp\downloads\new folder\lib\site-pac
kages (1.21.5)
Requirement already satisfied: scikit-learn in c:\users\hp\downloads\new folder\lib\s
ite-packages (1.0.2)
Requirement already satisfied: xgboost in c:\users\hp\downloads\new folder\lib\site-p
ackages (2.1.4)
Requirement already satisfied: matplotlib in c:\users\hp\downloads\new folder\lib\sit
e-packages (3.5.2)
Requirement already satisfied: seaborn in c:\users\hp\downloads\new folder\lib\site-p
ackages (0.11.2)
Requirement already satisfied: joblib in c:\users\hp\downloads\new folder\lib\site-pa
ckages (1.1.0)
Requirement already satisfied: openpyxl in c:\users\hp\downloads\new folder\lib\site-
packages (3.0.10)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\downloads\new folder\lib\s
ite-packages (from pandas) (2022.1)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\hp\downloads\new fo
lder\lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: scipy>=1.1.0 in c:\users\hp\downloads\new folder\lib\s
ite-packages (from scikit-learn) (1.9.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\hp\downloads\new fold
er\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: pillow>=6.2.0 in c:\users\hp\downloads\new folder\lib
\site-packages (from matplotlib) (9.2.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\hp\downloads\new folder
\lib\site-packages (from matplotlib) (1.4.2)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\downloads\new folder\li
b\site-packages (from matplotlib) (21.3)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\downloads\new folder
\lib\site-packages (from matplotlib) (4.25.0)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\hp\downloads\new folder\l
ib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\downloads\new folder\lib\s
ite-packages (from matplotlib) (0.11.0)
Requirement already satisfied: et_xmlfile in c:\users\hp\downloads\new folder\lib\sit
e-packages (from openpyxl) (1.1.0)
Requirement already satisfied: six>=1.5 in c:\users\hp\downloads\new folder\lib\site-
packages (from python-dateutil>=2.8.1->pandas) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

In [21]:
```
df['RetentionRate']=1-df['ChurnRate']
```

In [22]:
```
df['CAC'] = (
    df["MarketingCost"]
    / df["Conversions"]
)
```

In [23]:
```
df["ROI_Percentage"]=df['Revenue']-df['CAC']/df['CAC']
```

FEATURE SELECTION

In [24]:
```
features = [
    "Leads",
    "Conversions",
    "Revenue",
    "MarketingCost",
    "CAC",
    "CLV",
    "RetentionRate",
    "CTR"
]

X = df[features]
y = df["ROI_Percentage"]
```

In [25]:
```
df.head()
```

Out[25]:

	CustomerID	Date	CampaignName	MarketingChannel	Product	Region	Impressions	Clic
0	CUST-100000	1/20/2025	Retention Drive	Youtube	Analytics Hub	Europe	33232	19
1	CUST-100001	1/1/2025	Retention Drive	Facebook Ads	Customer Insights Platform	North America	47933	34
2	CUST-100002	2/17/2025	Black Friday Promo	Instagram	CRM Pro	Asia Pacific	16504	21
3	CUST-100003	6/25/2025	High Intent Leads	Instagram	Marketing Automation Suite	Africa	84979	114
4	CUST-100004	8/11/2025	High Intent Leads	Instagram	Customer Insights Platform	Middle East	139414	217

TRAIN TEST SPLIT

In [26]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
```

```
    random_state=42
)
```

TRAIN XGBOOST MODEL

In [27]:
```
from xgboost import XGBRegressor

model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6,
    random_state=42
)

model.fit(X_train, y_train)
```

Out[27]:
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=0.05, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=6, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=200, n_jobs=None,
             num_parallel_tree=None, random_state=42, ...)
```

MODEL EVALUATION

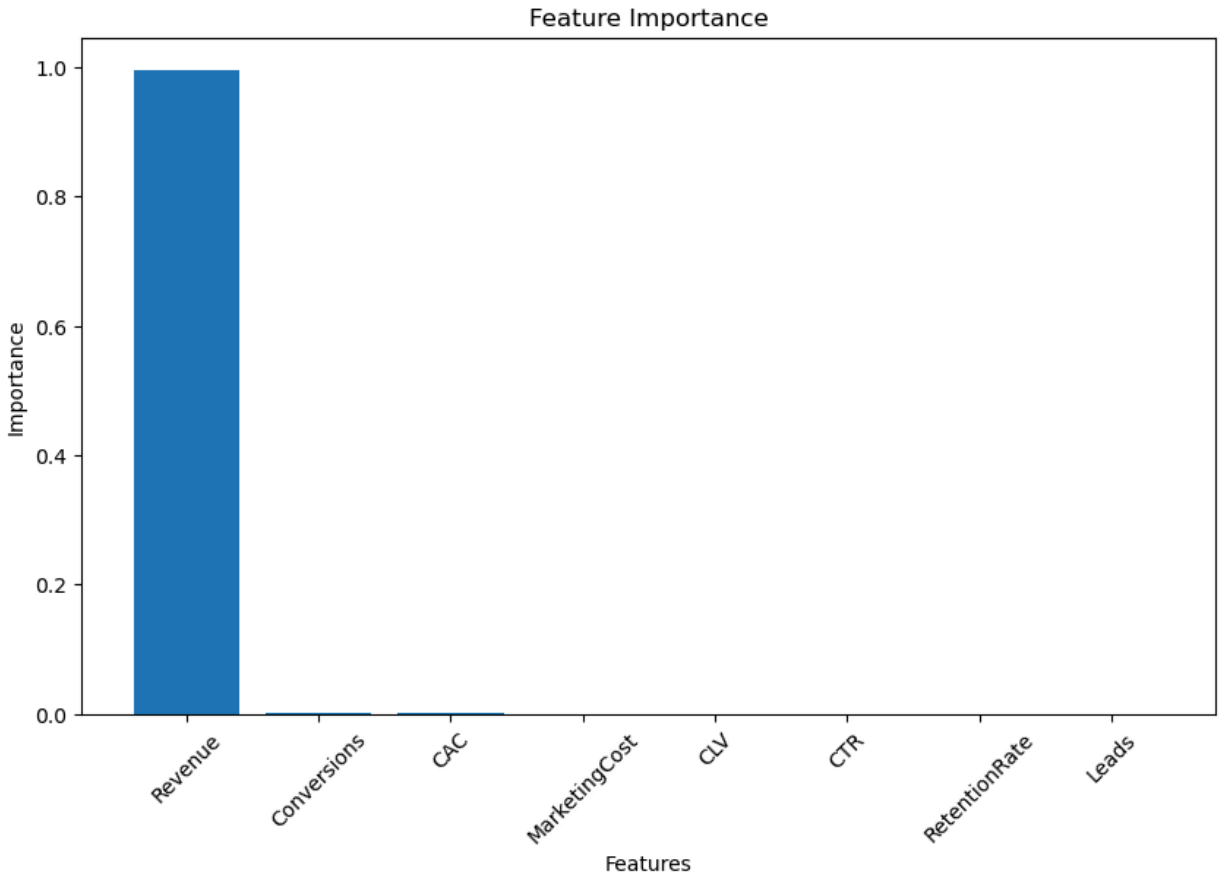
In [28]:
```
from sklearn.metrics import mean_absolute_error, r2_score

predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Mean Absolute Error:", mae)
print("R2 Score:", r2)
```

```
Mean Absolute Error: 2206.747682893361
R2 Score: 0.9930981744392507
```

FEATURE SELECTION

In [29]:
```
import matplotlib.pyplot as plt

importance = model.feature_importances_

feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": importance
})

feature_importance = feature_importance.sort_values(
```

```
    by="Importance",
    ascending=False
)

plt.figure(figsize=(10,6))
plt.bar(feature_importance["Feature"], feature_importance["Importance"])
plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.xticks(rotation=45)
plt.show()
```



SAVE MODEL USING JOBLIB

In [30]:
```
import joblib

joblib.dump(model, "roi_prediction_model.pkl")

print("Model saved successfully.")
```

```
Model saved successfully.
```

TKINTER GUI APPLICATION

This GUI allows users to:

input campaign metrics, predict ROI, estimate campaign profitability.

FULL TKINTER GUI CODE

In [ ]:
```
import tkinter as tk
from tkinter import messagebox
import numpy as np
import joblib

# Load trained model
model = joblib.load("roi_prediction_model.pkl")

# Create main window
root = tk.Tk()
root.title("Marketing ROI Prediction App")
root.geometry("500x650")
root.configure(bg="#f2f2f2")

# Title

title = tk.Label(
    root,
    text="Marketing Campaign ROI Predictor",
    font=("Arial", 18, "bold"),
    bg="#f2f2f2"
)

title.pack(pady=20)

# Input fields
```

In [31]:
```
import tkinter as tk
from tkinter import messagebox
import numpy as np
import joblib

# Load trained model
model = joblib.load("roi_prediction_model.pkl")

# Create main window
root = tk.Tk()
root.title("Marketing ROI Prediction App")
root.geometry("500x650")
root.configure(bg="#f2f2f2")

# Title

title = tk.Label(
    root,
    text="Marketing Campaign ROI Predictor",
    font=("Arial", 18, "bold"),
    bg="#f2f2f2"
)

title.pack(pady=20)
fields = [
    "Leads",
    "Conversions",
    "Revenue",
```

```
    "MarketingCost",
    "CAC",
    "CLV",
    "RetentionRate",
    "CTR"
]

entries = {}

for field in fields:
    frame = tk.Frame(root, bg="#f2f2f2")
    frame.pack(pady=5)
    label = tk.Label(
        frame,
        text=field,
        width=20,
        anchor="w",
        bg="#f2f2f2",
        font=("Arial", 11)
    )
    label.pack(side=tk.LEFT)
    entry = tk.Entry(frame, width=20)
    entry.pack(side=tk.RIGHT)

    entries[field] = entry

# Prediction function

def predict_roi():
    try:
        values = []

        for field in fields:
            value = float(entries[field].get())
            values.append(value)

        input_data = np.array(values).reshape(1, -1)
        prediction = model.predict(input_data)[0]

        result_label.config(
            text=f"Predicted ROI: {prediction:.2f}%"
        )

        # Business interpretation
        if prediction >= 40:
            recommendation = "Excellent campaign profitability. Scale investment."
        elif prediction >= 15:
            recommendation = "Moderately profitable campaign. Optimize performance."

        else:
            recommendation = "Low profitability risk. Reduce or reassess spending."

        recommendation_label.config(
            text=recommendation
        )

    except ValueError:
        messagebox.showerror(
            "Input Error",
            "Please enter valid numeric values."
```

```
            )

# Predict button

predict_button = tk.Button(
    root,
    text="Predict ROI",
    command=predict_roi,
    font=("Arial", 12, "bold"),
    bg="#4CAF50",
    fg="white",
    padx=10,
    pady=5
)

predict_button.pack(pady=20)

# Results

result_label = tk.Label(
    root,
    text="",
     font=("Arial", 16, "bold"),
    bg="#f2f2f2",
    fg="blue"
)

result_label.pack(pady=10)

recommendation_label = tk.Label(
    root,
    text="",
    font=("Arial", 11),
    wraplength=400,
    bg="#f2f2f2",
    fg="green"
)

recommendation_label.pack(pady=10)

# Run application

root.mainloop()
```

In [ ]: